SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

PRESENTATION & VIVA VOCE DEFENSE GUIDE

Smart Home Monitoring System (IoT & CCTV Analysis) | 15-Minute Two-Student Examination Strategy

1. Synchronized 15-Minute Two-Student Presentation Strategy

Time Window	Lead Speaker	Slides & Topic	Key Content to Emphasize
00:00 – 03:30	Student 1 (Venkata Rao D.)	**Slides 1 – 5:** Introduction & Problem	Highlight visual ambiguity in standalone sensors and why CCTV integration is crucial.
03:30 – 07:00	Student 2 (Sivanesan)	**Slides 6 – 10:** Features & Architecture	Explain the 4-layer architecture, hardware specs (ESP32/Pi), and DHT11/PIR/LDR sensors.
07:00 – 10:30	Student 1 (Venkata Rao D.)	**Slides 11 – 14:** Software & Block Diagram	Walk through the Next.js/React stack, optical motion pipeline, and system block diagram.
10:30 – 13:30	Student 2 (Sivanesan)	**Slides 15 – 18:** Demonstration Results	Explain block diagram components, live telemetry results, and 24/7 CCTV surveillance HUD.
13:30 – 15:00	Both Students (Joint Defense)	**Slides 19 – 20:** References & Viva Q&A	Summarize findings, present 12 IEEE references, and address examiner technical questions.

2. Top 16 Technical Viva Voce Questions & Authoritative Answers

Q1: Why did you choose MQTT over HTTP for sensor telemetry transmission in an IoT environment?

Answer: HTTP is a request-response protocol imposing substantial overhead (200–800 bytes per HTTP header), requiring continuous polling which wastes network bandwidth and edge battery power. In contrast, MQTT operates on an asynchronous Publish/Subscribe model with ultra-lightweight 2-byte packet headers. It maintains a persistent TCP connection with keep-alive pings, enabling edge sensors to publish readings only when new data is acquired. Furthermore, MQTT brokers handle topic filtering and one-to-many multicast distribution effortlessly.

Q2: What are MQTT Quality of Service (QoS) levels, and which level did your architecture adopt?

Answer: MQTT defines three QoS levels: QoS 0 ('At most once') transmits packets without acknowledgment (best effort); QoS 1 ('At least once') guarantees delivery via PUBACK packet handshakes but permits duplicate delivery upon network timeout; QoS 2 ('Exactly once') guarantees delivery without duplicates via a four-step handshake (PUBLISH, PUBREC, PUBREL, PUBCOMP), which incurs latency. Our architecture adopts QoS 1 for sensor telemetry and alert notifications, ensuring that critical alarms are never lost over transient wireless connections while avoiding the heavy handshake latency of QoS 2.

Q3: How does your application satisfy the assignment requirement when physical sensors are unavailable?

Answer: The professor's assignment guidelines state: 'You can generate in the software itself the required sensor data if you don’t have the hardware.' To satisfy this with high academic rigor, we engineered a mathematical diurnal simulation engine in src/lib/mock-data.ts. The engine models solar heating curves using sinusoidal equations (peaking at 15:00), inverse relative humidity shifts, solar lux illumination profiles, and Gaussian perturbation. When physical hardware is connected, the same REST API and MQTT ingestion pipelines process actual ADC and GPIO signals without requiring any frontend code changes.

Q4: How is optical CCTV feed analysis integrated with physical IoT sensor telemetry?

Answer: The system integrates optical CCTV and IoT sensors through an event-driven correlation pipeline. When an environmental sensor (such as an HC-SR501 PIR motion sensor or MQ-2 gas transducer) triggers an anomaly threshold, an event is dispatched to the central hub. The hub correlates the sensor's physical room location (e.g., 'backyard') and directs the web dashboard to highlight the corresponding CCTV feed, triggering optical motion analysis and alerting the user with both quantitative telemetry and visual verification simultaneously.

Q5: What algorithm is employed for motion detection in the CCTV video processing pipeline?

Answer: The video processing pipeline uses optical frame differencing combined with background subtraction (MOG2 Gaussian Mixture Models). Consecutive frames are captured at 30 FPS, converted to grayscale, downsampled to 640x360 pixels, and smoothed using a 5x5 Gaussian blur kernel to remove sensor noise. The absolute difference matrix $\Delta I = |I(t) - I(t-1)|$ is computed and binarized via Otsu's adaptive thresholding. Morphological dilation fills small gaps, and contours exceeding a threshold area of 500 pixels trigger motion bounding boxes and alert flags.

Q6: What are the primary hardware trade-offs between ESP32 and Raspberry Pi 4 in edge IoT deployments?

Answer: The ESP32 is a low-cost ($4–$6), ultra-low-power microcontroller (drawing ~15–150 mA) featuring instant boot times, real-time deterministic FreeRTOS execution, and integrated Wi-Fi/Bluetooth, making it ideal for battery-operated sensor nodes. However, it lacks sufficient RAM and compute power for high-definition video processing. Conversely, the Raspberry Pi 4 is a single-board computer ($45–$75) with a quad-core 1.5 GHz ARM CPU, up to 8 GB RAM, and a full Linux OS, capable of running OpenCV and YOLO object detection models, but draws significant power (5–15W) and requires cooling.

Q7: How does Next.js 14 App Router benefit an IoT web server application?

Answer: Next.js 14 App Router provides several advantages: (1) It unifies frontend UI components and backend serverless REST API route handlers within a single project, eliminating separate server maintenance; (2) React Server Components (RSC) enable server-side pre-rendering of initial sensor telemetry, minimizing client bundle sizes and accelerating initial page load; (3) Built-in edge runtime optimizations route API requests to globally distributed edge nodes near the client, reducing roundtrip latency below 50ms.

Q8: How does the system handle security, device authentication, and video stream privacy?

Answer: IoT security is implemented across multiple layers: (1) Transport Security: All MQTT and HTTP transmissions are encrypted using TLS 1.3 / HTTPS, preventing man-in-the-middle eavesdropping; (2) Authentication: Sensor nodes authenticate with the MQTT broker using unique device credentials and client certificates; (3) Web Server Protection: API route handlers validate incoming JSON schemas and sanitize actuator commands; (4) Video Privacy: Live CCTV streams are routed through secure tokens and obfuscated URL proxies rather than exposing open camera IP addresses.

Q9: What role does Edge Computing play in this architecture versus Cloud Computing?

Answer: The architecture implements a hybrid edge-cloud paradigm. Edge nodes (ESP32/Pi) execute high-frequency sensor signal sampling, ADC calibration, analog filtering, and local motion frame differencing locally. Only processed telemetry packets and detected anomaly alerts are published to the cloud web server. This edge intelligence drastically conserves wide-area network bandwidth, guarantees real-time local actuator responses (< 5ms) even if internet connectivity drops, and protects user privacy.

Q10: How do you mitigate false positives in PIR motion sensors and CCTV motion detection?

Answer: PIR sensors are susceptible to false alarms from sudden sunlight changes or air conditioning drafts, while optical motion detection can trigger on wind-blown curtains or shadows. Our system mitigates false positives through cross-modal sensor fusion: a high-priority intruder alarm is only escalated if both the physical PIR sensor AND optical frame differencing detect motion simultaneously within a 2-second time window in the same zone.

Q11: How would you scale this architecture to support thousands of smart homes?

Answer: Scaling involves three architectural enhancements: (1) Distributed MQTT Clustering: Deploy an enterprise EMQX or HiveMQ cluster behind a Layer-4 load balancer (HAProxy), supporting millions of concurrent persistent connections; (2) Time-Series Database: Replace in-memory state with an optimized time-series database (TimescaleDB or InfluxDB) with data retention and compression policies; (3) Serverless Edge Caching: Utilize Vercel/Cloudflare edge workers to cache read-heavy telemetry dashboards near end-users.

Q12: How does the bidirectional actuator feedback loop prevent state divergence?

Answer: In IoT actuation (e.g., toggling a light relay or unlocking a door), sending a command does not guarantee physical execution if the device is offline. Our system implements optimistic UI updates paired with closed-loop acknowledgment: when the user toggles a switch, the UI indicates a pending state; the edge actuator executes the physical relay toggle and publishes a confirmed state packet back via /api/controls. If no acknowledgment arrives within 3 seconds, the UI rolls back and notifies the user.

Q13: Explain the role of ADC and calibration in physical DHT11 and LDR sensors.

Answer: The LDR produces an analog voltage that varies with ambient light resistance. The microcontroller's Analog-to-Digital Converter (ADC), such as the ESP32's 12-bit SAR ADC, converts this voltage into a discrete digital value (0 to 4095). Calibration curves map the digital reading to physical lux units (E = (V_in - V_out) / R_fixed). For DHT11, a dedicated single-wire digital protocol transfers 40-bit data packets comprising integer and decimal bytes for relative humidity and temperature, validated using a checksum byte.

Q14: What database would you choose for production historical logging and why?

Answer: For production smart home logging, InfluxDB or TimescaleDB (PostgreSQL time-series extension) is ideal. Traditional relational databases (MySQL) degrade significantly under high-frequency time-stamped inserts. Time-series databases utilize Log-Structured Merge (LSM) trees, automatic chunk-based partitioning by timestamp, continuous aggregations (calculating 1-minute, 1-hour rollups), and delta-of-delta compression, reducing disk storage footprint by over 90%.

Q15: How does the system ensure fault tolerance if a sensor node disconnects?

Answer: The MQTT broker implements Last Will and Testament (LWT) messages. When an edge sensor node connects, it registers an LWT message on topic home/{room_id}/status with payload 'offline'. If the node crashes or loses power, the broker detects the missing keepalive ping and automatically broadcasts the LWT message. The web dashboard immediately flags the node as disconnected, updates the health badge to amber/red, and logs a warning alert.

Q16: How is the Next.js server configured for deployment on Vercel?

Answer: The project includes a dedicated vercel.json configuration defining framework presets and edge routing. Because Next.js is developed by Vercel, the App Router route handlers (/api/*) are automatically compiled into serverless and edge functions distributed across global points of presence (PoPs), delivering low-latency response times without manual container or Kubernetes configuration.